

NEU – DS&AI

Data Structures and Algorithms in Python

Lecture 5: Recursion, Recurrences, and Master Theorem

Faculty of Data Science and Artificial Intelligence

National Economics University

Lecture 5: Table of Contents

- 1 Recursive Thinking
- 2 From Recursion to Recurrence Relations
- 3 Master Theorem
- 4 Extended Master Theorem
- 5 Beyond the Standard Master Theorem
- 6 Practice and Key Takeaways

SECTION 1

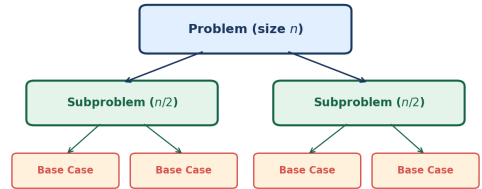
Recursive Thinking

Why Recursion?

Motivation

Many algorithms solve a problem by reducing it to one or more **smaller problems of the same type**.

- search in one half of a sorted array;
- sort the left half and right half, then merge;
- solve smaller subtrees in a tree;
- explore smaller states in dynamic programming.



Key idea

Recursion is natural when the problem has a **self-similar structure**.

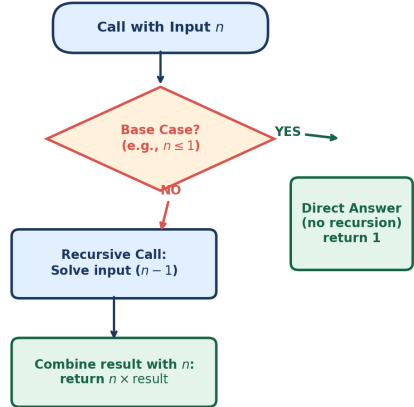
Anatomy of a Recursive Algorithm

Every recursive algorithm needs

- 1 **Base case:** solve a small input directly.
- 2 **Recursive case:** reduce to smaller input(s).
- 3 **Progress measure:** strictly decreases.
- 4 **Combination rule:** combine smaller answers.

Without a base case

The function calls itself forever and never returns.



Example 1: Factorial

```
1 FUNCTION FACTORIAL(n)
2 INPUT: non-negative integer n
3 OUTPUT: n!
4   IF n == 0 THEN
5     RETURN 1
6   RETURN n * FACTORIAL(n - 1)
```

Mathematical definition

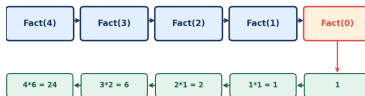
$$n! = \begin{cases} 1, & n = 0, \\ n \cdot (n - 1)!, & n > 0. \end{cases}$$

Running time

One recursive call and constant extra work per level:

$$T(n) = T(n - 1) + \Theta(1) = \Theta(n).$$

Call Chain & Return Values for FACTORIAL(4)

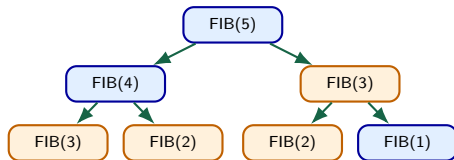


Example 2: Naive Fibonacci

```
1 FUNCTION FIB(n)
2 INPUT: non-negative integer n
3 OUTPUT: the n-th Fibonacci number
4 IF n <= 1 THEN
5     RETURN n
6 RETURN FIB(n - 1) + FIB(n - 2)
```

Why this is expensive

The same subproblems are recomputed many times, such as FIB(3), FIB(2), and FIB(1).



Repeated subproblems are highlighted.

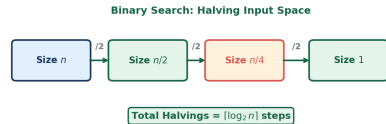
Recurrence

$$T(n) = T(n - 1) + T(n - 2) + \Theta(1),$$

which grows exponentially.

Example 3: Binary Search

```
1 FUNCTION BINARY_SEARCH(A, x, left, right)
2 INPUT: sorted array A and search value x
3 OUTPUT: index of x, or NONE
4     IF left > right THEN
5         RETURN NONE
6     mid = (left + right) DIV 2
7     IF A[mid] == x THEN
8         RETURN mid
9     ELSEIF x < A[mid] THEN
10        RETURN BINARY_SEARCH(A, x, left, mid - 1)
11    ELSE
12        RETURN BINARY_SEARCH(A, x, mid + 1, right)
```



Running time

One recursive call on half the input:

$$T(n) = T(n/2) + \Theta(1) = \Theta(\log n).$$

Example 4: Merge Sort

```
1 FUNCTION MERGE_SORT(A)
2 INPUT: array A of length n
3 OUTPUT: sorted array containing all elements of A
4     IF LENGTH(A) <= 1 THEN
5         RETURN A
6     mid = LENGTH(A) DIV 2
7     left = MERGE_SORT(A[0 .. mid - 1])
8     right = MERGE_SORT(A[mid .. n - 1])
9     RETURN MERGE(left, right)
```

Three-step structure

- 1 **Divide:** split array into two halves.
- 2 **Conquer:** recursively sort both halves.
- 3 **Combine:** merge two sorted arrays into one sorted result.

Recurrence

$$T(n) = 2T(n/2) + \Theta(n).$$

SECTION 2 _____

From Recursion to Recurrence Relations _____

What is a Recurrence Relation?

Definition

A **recurrence relation** expresses the running time for input size n using running times of smaller input sizes.

Typical forms

$$T(n) = T(n - 1) + \Theta(1)$$

$$T(n) = T(n/2) + \Theta(1)$$

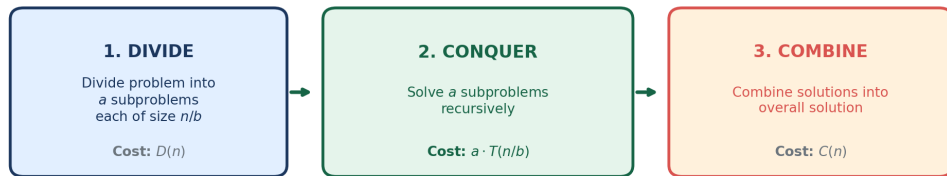
$$T(n) = 2T(n/2) + \Theta(n)$$

Goal of analysis

The recurrence describes the algorithm structure. We still need to solve it to find a simple growth class such as

$$\Theta(n), \quad \Theta(\log n), \quad \Theta(n \log n), \quad \Theta(2^n).$$

How to Build a Recurrence



Template for divide-and-conquer

$$T(n) = aT(n/b) + f(n)$$

- a : number of subproblems;
- n/b : size of each subproblem;
- $f(n)$: divide, combine, and other work.

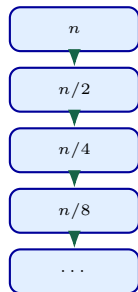
Do not forget the base case

Usually $T(1) = \Theta(1)$ or $T(n) = \Theta(1)$ for $n \leq n_0$.

Examples: Algorithm Structure to Recurrence

Algorithm	Recursive structure	Recurrence
Factorial	one call on $n - 1$; constant work	$T(n) = T(n - 1) + \Theta(1)$
Binary Search	one call on $n/2$; constant work	$T(n) = T(n/2) + \Theta(1)$
Merge Sort	two calls on $n/2$; linear merge	$T(n) = 2T(n/2) + \Theta(n)$
Naive Fibonacci	calls on $n - 1$ and $n - 2$; constant work	$T(n) = T(n - 1) + T(n - 2) + \Theta(1)$

Recursion Tree: Binary Search



Cost per level

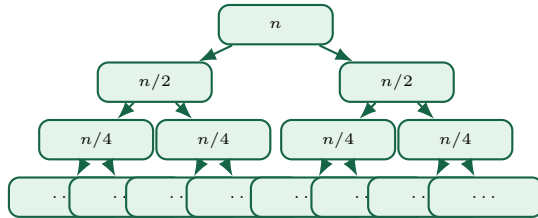
At each level, there is one subproblem and $\Theta(1)$ non-recursive work.

Number of levels

Stop when $n/2^k = 1$, so $k = \log_2 n$.

$$T(n) = \Theta(\log n).$$

Recursion Tree: Merge Sort



Level cost

Level	Subproblems	Total merge cost
0	1 of size n	n
1	2 of size $n/2$	n
2	4 of size $n/4$	n

Conclusion

$\log_2 n$ levels, each costs $\Theta(n)$:

$$T(n) = \Theta(n \log n).$$

Activity: Build the Recurrence

Student task

For each recursive algorithm pattern, write a recurrence and predict the growth class.

	Algorithm pattern	Recurrence
1	One recursive call on $n - 1$ and $\Theta(n)$ work outside	
2	One recursive call on $n/2$ and $\Theta(n)$ work outside	
3	Two recursive calls on $n/2$ and $\Theta(1)$ work outside	
4	Four recursive calls on $n/2$ and $\Theta(n)$ work outside	

SECTION 3

Master Theorem

Master Theorem: Standard Form

Recurrence form

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$, $b > 1$, and $f(n)$ is non-negative for sufficiently large n .

The critical comparison

$$n^{\log_b a}$$

This term represents the growth generated by the recursive branching structure.

Intuition

Compare **recursive work** $n^{\log_b a}$ against **extra work** $f(n)$. The larger component usually determines the final complexity.

Master Theorem: Three Cases

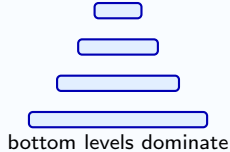
Case	Condition	Result
1	If $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
2	If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for $k \geq 0$	$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
3	If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ and $af(n/b) \leq cf(n)$ for some $c < 1$	$T(n) = \Theta(f(n))$

Memory aid

Recursive work dominates $\Rightarrow$ balanced $\Rightarrow$ non-recursive work dominates.

Visual Intuition for the Three Cases

Case 1: leaves



$$T(n) = \Theta(n^{\log_b a})$$

Case 2: balanced



$$T(n) = \Theta(n^{\log_b a} \log n)$$

Case 3: root



$$T(n) = \Theta(f(n))$$

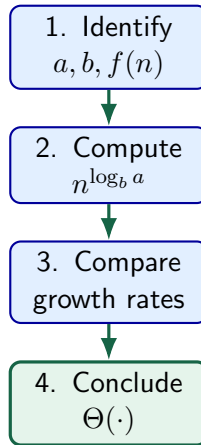
Procedure for Applying Master Theorem

Four steps

- 1 Identify a , b , and $f(n)$.
- 2 Compute $n^{\log_b a}$.
- 3 Compare $f(n)$ with $n^{\log_b a}$.
- 4 Select the matching case and conclude $T(n)$.

Important

Do not look only at the number of subproblems. The combine cost $f(n)$ can change the answer.



Example: Merge Sort via Master Theorem

Recurrence

$$T(n) = 2T(n/2) + \Theta(n)$$

Step-by-step

- $a = 2, b = 2, f(n) = \Theta(n)$.
- $n^{\log_b a} = n^{\log_2 2} = n$.
- $f(n) = \Theta(n)$ matches the critical term.
- Balanced case with $k = 0$.

Conclusion

$$T(n) = \Theta(n \log n).$$

The extra $\log n$ factor appears because there are $\log n$ levels, each with total cost $\Theta(n)$.

Example: Recursive Work Dominates

Recurrence

$$T(n) = 8T(n/2) + n^2$$

Analysis

- $a = 8, b = 2, f(n) = n^2$.
- $n^{\log_2 8} = n^3$.
- $n^2 = O(n^{3-\varepsilon})$ with $\varepsilon = 1$.

Case 1

Recursive branching is stronger than the work outside recursive calls.

$$T(n) = \Theta(n^3).$$

Example: Non-recursive Work Dominates

Recurrence

$$T(n) = 2T(n/2) + n^2$$

Analysis

- $a = 2, b = 2, f(n) = n^2$.
- $n^{\log_2 2} = n$.
- $n^2 = \Omega(n^{1+\varepsilon})$ with $\varepsilon = 1$.
- Regularity: $2(n/2)^2 = n^2/2 \leq cn^2$ for $c = 1/2$.

Case 3

Extra work outside recursion dominates.

$$T(n) = \Theta(n^2).$$

Quick Check: Which Case?

Exercise

For each recurrence, identify a , b , $f(n)$, the Master Theorem case, and the final bound.

	Recurrence	Case	Result
1	$T(n) = 3T(n/2) + n^2$		
2	$T(n) = 4T(n/2) + n^2$		
3	$T(n) = 16T(n/4) + n$		
4	$T(n) = 3T(n/2) + n$		

SECTION 4

Extended Master Theorem

Extended Form: $f(n) = \Theta(n^k \log^p n)$

Common exercise form

$$T(n) = aT(n/b) + \Theta(n^k \log^p n).$$

Shortcut comparison

Instead of comparing $f(n)$ with $n^{\log_b a}$ directly, compare

a with b^k .

Why it works

The critical exponent is $\log_b a$. Comparing a and b^k is equivalent to comparing $\log_b a$ and k .

Extended Master Theorem: Summary Table

Condition	Result
$a > b^k$	$T(n) = \Theta(n^{\log_b a})$
$a = b^k, p > -1$	$T(n) = \Theta(n^k \log^{p+1} n)$
$a = b^k, p = -1$	$T(n) = \Theta(n^k \log \log n)$
$a = b^k, p < -1$	$T(n) = \Theta(n^k)$
$a < b^k, p \geq 0$	$T(n) = \Theta(n^k \log^p n)$
$a < b^k, p < 0$	typically $O(n^k)$; tight bound requires additional verification

Teaching note

The boundary $p = -1$ produces the special term $\log \log n$.

Example: $T(n) = 2T(n/2) + n \log n$

Identify parameters

$$a = 2, \quad b = 2, \quad f(n) = n \log n.$$

Here $f(n) = \Theta(n^k \log^p n)$ with

$$k = 1, \quad p = 1.$$

Compare

$$a = 2, \quad b^k = 2^1 = 2.$$

Balanced case: $a = b^k$ and $p > -1$.

Conclusion

$$T(n) = \Theta(n \log^2 n).$$

Example: Boundary Case $p = -1$

Recurrence

$$T(n) = 2T(n/2) + \frac{n}{\log n}.$$

Parameters

$$a = 2, \quad b = 2, \quad k = 1, \quad p = -1.$$

Since $a = b^k$, this is a balanced boundary case.

Conclusion

$$T(n) = \Theta(n \log \log n).$$

This result is slower than $\Theta(n)$ but faster than $\Theta(n \log n)$.

Representative Examples

	Recurrence	Result	Remark
1	$T(n) = 3T(n/2) + n^2$	$\Theta(n^2)$	outside work dominates
2	$T(n) = 4T(n/2) + n^2$	$\Theta(n^2 \log n)$	balanced
3	$T(n) = 4T(n/2) + \log n$	$\Theta(n^2)$	recursion dominates
4	$T(n) = 2T(n/2) + n \log n$	$\Theta(n \log^2 n)$	$p = 1$
5	$T(n) = 2T(n/2) + n / \log n$	$\Theta(n \log \log n)$	$p = -1$
6	$T(n) = 3T(n/2) + n$	$\Theta(n^{\log_2 3})$	recursion dominates

Quick Check: Extended Form

Exercise

Solve each recurrence using the extended form when possible.

	Recurrence	Compare	Result
1	$T(n) = 2T(n/2) + n \log^2 n$		
2	$T(n) = 2T(n/2) + n / \log^2 n$		
3	$T(n) = 6T(n/3) + n^2 \log n$		
4	$T(n) = 3T(n/4) + n \log n$		

SECTION 5

Beyond the Standard Master Theorem

When Master Theorem Cannot Be Applied Directly

Not in standard form

- non-constant number of subproblems:
 $2^n T(n/2) + n^n$;
- non-uniform subproblem sizes:
 $T(n/3) + T(2n/3) + n$;
- subtract-and-conquer:
 $T(n) = T(n - 1) + n$;
- negative $f(n)$ or failed regularity condition;
- coefficient $a < 1$ in the standard theorem.

Alternative tools

- recursion tree;
- substitution and induction;
- direct summation;
- variable transformation;
- Akra–Bazzi theorem for unequal splits.

Subtract-and-Conquer Recurrences

Typical form

$$T(n) = aT(n - b) + f(n).$$

The input size decreases by a fixed amount, not by a constant fraction.

Recurrence	Result
$T(n) = T(n - 1) + 1$	$\Theta(n)$
$T(n) = T(n - 1) + n$	$\Theta(n^2)$
$T(n) = 2T(n - 1) + 1$	$\Theta(2^n)$

Main idea

Expand the recurrence until reaching the base case, then sum the work across levels.

Example: $T(n) = T(n - 1) + n$

Expansion

$$\begin{aligned}T(n) &= T(n - 1) + n \\&= T(n - 2) + (n - 1) + n \\&= T(n - 3) + (n - 2) + (n - 1) + n \\&\dots \\&= T(1) + 2 + 3 + \dots + n.\end{aligned}$$

Summation

$$1 + 2 + \dots + n = \frac{n(n + 1)}{2} = \Theta(n^2).$$

Conclusion

$$T(n) = \Theta(n^2).$$

Guess-and-Prove Method

Method

When no direct theorem applies, guess the solution form and prove it by mathematical induction.

- 1 Observe the recurrence and guess a growth class.
- 2 Assume the hypothesis holds for smaller inputs.
- 3 Substitute the hypothesis into the recurrence.
- 4 Verify the inequality for sufficiently large n .

If the proof fails

The guess may need stronger constants, a correction term, or a different growth class.

Common use

Proving upper bounds such as $T(n) \leq cn \log n$ or lower bounds such as $T(n) \geq cn$.

Example Beyond Master Theorem

Recurrence

$$T(n) = \sqrt{n} T(\sqrt{n}) + n.$$

Standard Master Theorem does not apply because the number of subproblems $\sqrt{n}$ is not constant.

Transformation

Let $U(n) = T(n)/n$. Then

$$U(n) = U(\sqrt{n}) + 1.$$

Arguments shrink as

$$n, \sqrt{n}, n^{1/4}, n^{1/8}, \dots$$

Number of levels

Stop when $n^{1/2^k}$ is constant, so
 $2^k = \Theta(\log n)$ and

$$k = \Theta(\log \log n).$$

Hence

$$T(n) = \Theta(n \log \log n).$$

SECTION 6

Practice and Key Takeaways

Solve the following recurrences

- ① $T(n) = 8T(n/2) + n^2$.
- ② $T(n) = 2T(n/2) + n \log^2 n$.
- ③ $T(n) = 2T(n/2) + n / \log n$.
- ④ $T(n) = 4T(n/2) + \log n$.
- ⑤ $T(n) = T(n-1) + n$.
- ⑥ $T(n) = \sqrt{n} T(\sqrt{n}) + n$.
- ⑦ Explain why $T(n) = T(n/3) + T(2n/3) + n$ does not fit the standard Master Theorem.

Self-Test Quiz

	Question	Answer
1	$T(n) = 4T(n/2) + n^2$	
2	$T(n) = 2T(n/2) + n/\log n$	
3	Which one does not fit standard Master Theorem: $2T(n/2) + n$, $4T(n/2) + n^2$, $T(n/3) + T(2n/3) + n$?	
4	$T(n) = T(n-1) + n$	
5	$T(n) = \sqrt{n}T(\sqrt{n}) + n$	

Common Mistakes

Mistake 1: Ignore $f(n)$

$T(n) = 2T(n/2) + n$ and
 $T(n) = 2T(n/2) + n^2$ have the same
recursive calls but different final complexities.

Mistake 2: Apply Master Theorem to any recurrence

Unequal splits, subtract-and-conquer, or
non-constant a require other methods.

Mistake 3: Forget base cases

Recursive algorithms must terminate;
recurrence relations usually assume
 $T(1) = \Theta(1)$.

Mistake 4: Confuse O and Θ

Master Theorem usually gives a tight Θ
bound, not only an upper bound.

Key Takeaways

- **Recursion** solves a problem by calling the same procedure on smaller inputs.
- Every recursive algorithm needs a **base case**, a **recursive case**, and a decreasing progress measure.
- A **recurrence relation** describes how running time depends on smaller input sizes.
- For divide-and-conquer recurrences $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$.
- Master Theorem has three main outcomes: recursive work dominates, balanced work, or non-recursive work dominates.
- The extended form $f(n) = \Theta(n^k \log^p n)$ handles many logarithmic variations.
- Not all recurrences fit the standard theorem; use recursion trees, summations, transformations, or induction when needed.